

Task Management System Report

1. Introduction

This report describes the development of **TaskManagementSystem**, a small ASP.NET Core MVC web application created for a university assignment.

The purpose of the application is to allow a signed-in user to manage assignable users and tasks. Each task can have a title, description, priority, creation date, and an assigned user.

The project was intentionally kept simple. It uses a dummy session-based login system instead of ASP.NET Identity, Entity Framework Core with SQLite for the database, and standard Razor views with Bootstrap for the user interface.

2. Project Overview

Project name: TaskManagementSystem

Application type: ASP.NET Core MVC Web Application

Framework: .NET 8

Database: SQLite

ORM: Entity Framework Core

Authentication: Simple session-based dummy login

Testing method: Manual testing checklist

The application supports the main requirements of a basic task management system:

- Registering and logging in users
 - Managing assignable users
 - Creating, editing, deleting, and viewing tasks
 - Assigning tasks to users
 - Sorting tasks by creation date or priority
 - Validating user input
 - Storing data in a SQLite database
-

3. Technologies Used

The following technologies were used in this project:

Technology	Purpose
ASP.NET Core MVC	Main web application framework
.NET 8	Runtime and development platform
Entity Framework Core	Database access and migrations
SQLite	Local database storage
Razor Views	Rendering HTML pages
Bootstrap	Basic styling and layout
C#	Main programming language

4. Project Structure

The project follows the standard ASP.NET Core MVC structure.

```
TaskManagementSystem/
  Controllers/
  Data/
  Migrations/
  Models/
  ViewModels/
  Views/
  wwwroot/
  appsettings.json
  Program.cs
```

```
docs/
  testing-checklist.md
```

Main folders

Folder	Description
<code>Controllers/</code>	Contains MVC controllers such as AccountController, UsersController, and TasksController
<code>Data/</code>	Contains <code>AppDbContext</code> , which manages database access
<code>Models/</code>	Contains the main data models

<code>ViewModels</code>	Contains view-specific models for forms
<code>/</code>	
<code>Views/</code>	Contains Razor pages for the user interface
<code>Migrations</code>	Contains Entity Framework database migrations
<code>/</code>	
<code>wwwroot/</code>	Contains static files such as CSS, JavaScript, Bootstrap, and jQuery
<code>docs/</code>	Contains project documentation and testing checklist

5. Data Models

The application uses four main models.

5.1 AppUser

`AppUser` represents a login account.

Main fields:

- `Id`
- `Username`
- `Password`

This model is used only for the dummy authentication system. Passwords are stored in plain text because the project uses a minimal login system for assignment purposes only.

5.2 User

`User` represents a person who can be assigned to a task.

Main fields:

- `Id`
- `Name`
- `Email`
- `Tasks`

Validation is applied to make sure that the name and email are required, and that the email has a valid format.

5.3 TaskItem

`TaskItem` represents a task in the system.

Main fields:

- `Id`
- `Title`
- `Description`
- `Priority`
- `CreatedAt`
- `UserId`
- `User`

Each task is assigned to one user. The `CreatedAt` value is set when the task is created and is not changed when the task is edited.

5.4 Priority

`Priority` is an enum used to represent the importance of a task.

Values:

- Low
- Medium
- High

In SQLite, the enum is stored as an integer.

6. Authentication

The authentication system is intentionally simple and session-based.

The application includes:

- Register
- Login
- Logout

When a user logs in successfully, the username is stored in the session using:

```
HttpContext.Session.SetString("Username", username);
```

Protected pages check whether the session contains a username. If no username exists in the session, the user is redirected to the login page.

A **BaseController** is used so that **UsersController** and **TasksController** can share the same login protection logic.

The shared layout also changes depending on the login state:

- Logged out users see **Login** and **Register**
 - Logged in users see **Hello, username** and **Logout**
-

7. User Management

The user management feature allows the signed-in user to manage people who can be assigned to tasks.

Implemented actions:

Action	Description
Index	Displays all users
Create	Adds a new user
Edit	Updates an existing user
Delete	Deletes a user after confirmation

Each form uses validation. If the submitted data is invalid, the same form is shown again with validation messages.

The edit and delete actions return **NotFound()** if the selected user does not exist.

8. Task Management

The task management feature allows the signed-in user to manage tasks.

Implemented actions:

Action	Description
Index	Displays all tasks
Create	Adds a new task
Edit	Updates an existing task
Delete	Deletes a task after confirmation

Each task includes:

- Title
- Description
- Priority
- Assigned user
- Creation date

The task form uses `TaskFormViewModel`. This allows the form to contain both task fields and the dropdown list of assignable users.

When creating or editing a task, the user can choose:

- Priority: Low, Medium, or High
- Assigned user: selected from existing users

When editing a task, only the editable fields are changed. The original creation date remains the same.

9. Sorting

The task list supports sorting using a `sortBy` query parameter.

Available sorting options:

Value	Result
<code>date_desc</code>	Newest first
<code>date_asc</code>	Oldest first
<code>priority_asc</code>	Low → Medium → High
<code>priority_desc</code>	High → Medium → Low

If no sorting option is selected, the default sorting is newest first.

The selected sorting option is preserved after the form is submitted.

10. Validation

Validation is applied using Data Annotations and `ModelState.IsValid`.

Examples of validation rules:

- Required fields
- Maximum text length
- Valid email format
- Password confirmation during registration

Validation is applied on:

- Register form
- Login form
- User create/edit forms
- Task create/edit forms

Razor validation helpers are used in the views:

```
<div asp-validation-summary="ModelOnly"></div>
<span asp-validation-for="FieldName"></span>
```

Client-side validation is also enabled using `_ValidationScriptsPartial`.

11. Database

The application uses SQLite as the database.

The database connection string is stored in `appsettings.json`:

Data Source=app.db

Entity Framework Core migrations are used to create the database structure.

The application also calls `Database.Migrate()` on startup. This means that the database is created automatically when the application starts if it does not already exist.

To reset the database, the `app.db` file can be deleted and the application can be run again.

12. Running the Application

To run the application locally, the following steps are used.

Step 1: Restore packages

```
dotnet restore
```

Step 2: Restore local tools

```
dotnet tool restore
```

Step 3: Apply database migrations

```
dotnet ef database update --project TaskManagementSystem
```

This step is optional because the application also runs migrations automatically on startup.

Step 4: Run the application

```
dotnet run --project TaskManagementSystem
```

The application runs locally at:

```
http://localhost:5078
```

13. Manual Testing

The project uses manual testing instead of automated tests.

The testing checklist is stored in:

docs/testing-checklist.md

The checklist covers:

- Registering a new account
- Logging in
- Logging out
- Creating users
- Editing users
- Deleting users
- Creating tasks
- Editing tasks
- Deleting tasks
- Sorting tasks
- Checking validation messages
- Building the project

The project build can be tested using:

dotnet build TaskManagementSystem/TaskManagementSystem.csproj

A successful build should show:

Build succeeded.

0 Warning(s)

0 Error(s)

14. Screenshots

The following screenshots should be included in the final submission:

- Register page
- Login page
- Users list
- Create user form
- Edit user form
- Tasks list with sorting dropdown
- Create task form
- Edit task form
- Delete confirmation page

The screenshots can be saved inside:

docs/screenshots/

Example screenshot reference:

![Tasks list](docs/screenshots/tasks-list.png)

If the application is deployed and served through Cloudflare, a screenshot showing the live hosted URL can also be included.

15. Version Control and Branching

The project was developed using Git and GitHub.

The codebase was organised using separate branches for different parts of the application.

Example branches:

Branch	Purpose
<code>main</code>	Stable final version
<code>development</code>	Main development branch
<code>feature/models-database</code>	Data models and database setup
<code>feature/users</code>	User CRUD feature
<code>feature/tasks</code>	Task CRUD feature
<code>feature/sorting-validation</code>	Sorting and validation logic

Each feature was developed separately and then merged back into `development`. After testing, the final version was merged into `main`.

This helped keep the project organised and made it easier to track changes.

16. Deployment / Production Testing

The application can be tested in a production-like environment after local development is complete.

The production testing stage should check that:

- The application starts correctly
- Registration works
- Login and logout work
- Users can be created, edited, and deleted
- Tasks can be created, edited, deleted, and sorted
- The database is created correctly
- Validation works correctly
- The application is accessible through the hosted URL
- Static files such as CSS and JavaScript load correctly

If the project is hosted behind Cloudflare, the final report can mention that Cloudflare is used for DNS and serving the public URL.

17. Limitations

The project is intentionally minimal and has some limitations:

- It does not use ASP.NET Identity
- Passwords are stored in plain text
- There are no automated tests
- There are no advanced user roles
- All signed-in users can access the same task and user management pages
- The database is SQLite, which is suitable for a small assignment project but not ideal for a large production system

These limitations are acceptable because the goal of the assignment is to demonstrate MVC structure, CRUD operations, validation, sorting, database usage, and basic authentication.

18. Conclusion

TaskManagementSystem successfully implements a small task management web application using ASP.NET Core MVC, .NET 8, Entity Framework Core, and SQLite.

The application includes user registration, login, logout, user management, task management, task assignment, sorting, validation, and database persistence. The project also follows a clear structure and uses Git branches to separate development stages.

Overall, the system meets the main assignment requirements and provides a functional foundation for a simple task management application.